Project 4: Double Descent in Linear Models

Giacomo Comitani

Historically in machine learning, models were constructed following the bias-variance trade-off, balancing under-fitting and over-fitting to obtain an optimal predictor that does not memorize the training data, but rather learns a general rule to perform well on unseen data. Classically, this implied restricting the number of features to be strictly less than the number of examples in the dataset, aiming to find the sweet spot at the minimum of the U-shaped test risk curve. In modern practice, however, highly complex models like neural networks routinely achieve optimal generalization results even when operating in an over-parameterized regime, strictly at or beyond the interpolation threshold (where training error is exactly zero). In their paper, Belkin et al. [1] reconcile this apparent contradiction by demonstrating the "double descent" risk curve. They observed that by increasing the model capacity beyond the interpolation point, predictors can achieve a second descent in test error, challenging the classical limitations of ML theory. This project aims to empirically reproduce this phenomenon using linear models. By generating a synthetic dataset with noisy linear labels, I follow their theoretical framework to observe the double descent curve and validate the mechanisms described by the authors

May 08, 2026

Contents

1. Initial Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

2. Generation of the Synthetic Dataset

The first step of the project was to construct the synthetic dataset. To prevent data leakage, I generated the training and test sets as completely independent blocks right from the start. As highlighted by the official scikit-learn guidelines [2], splitting the data into independent training and test subsets must always be the very first step before any processing. This strict separation guarantees that the model has no possibility of accessing the test data during the training phase.

To simulate a real-world scenario and reproduce the classical U-shaped bias-variance trade-off, I designed the synthetic dataset such that the true target values are calculated using only the first 20 features out of the available 205. By iteratively training the model on an increasing number of features d, I was able to observe both the *underfitting* and *overfitting* regimes, fully confirming classical statistical learning theory. For the main experiments (OLS, Ridge, and Double Descent), I used a default noise level of $\sigma = 1.0$.

Since the assignment gave us the freedom to design the data-generating process, I chose to sample the input features X from a standard Gaussian distribution. I made this choice because it naturally gives all the features the exact same scale (mean 0 and variance 1) and keeps them independent. This way, I don't need to manually normalize the data. Following the assignment guidelines, I defined the target labels as a linear combination of these inputs:

$$y = Xw^* + \varepsilon \tag{1}$$

where:

- $y \in \mathbb{R}^n$ represents the vector of the target labels.
- $X \in \mathbb{R}^{n \times d}$ is the matrix containing the input features, generated by sampling from a standard Gaussian distribution.
- $w^* \in \mathbb{R}^d$ is the vector of the weights.
- $\varepsilon \in \mathbb{R}^n$ is the noise vector, sampled from a Gaussian distribution with mean zero and standard deviation σ to introduce stochasticity into the observations.

During the creation of the labels, I introduced noise to simulate a realistic scenario. This noise was again sampled from a Gaussian distribution. To prevent data leakage, the noise vectors for the training and the test sets were generated via two separate calls. Reusing the same noise for training and test would violate the i.i.d. assumption, creating a situation in which test labels are mathematically correlated with training labels [3].

Finally, I implemented a fixed seed for the random number generator (seed=30). A core requirement of this project is reproducibility; the seed acts as a deterministic starting point for the pseudo-random numbers, ensuring that the exact same dataset and results can be perfectly replicated multiple times and on different machines [4]. Without the seed, the process of generating random numbers would initialize at a different state during every execution (typically using the machine's

internal clock as a starting point), making it impossible to obtain a perfect replica of the work on different machines.

3. Model Training

In this section of the report, I will cover the main theoretical concepts that guided the development of the project. As requested by the assignment guidelines, I focused on *Least Squares* and *Ridge Regression* to train the linear models.

3.1. Least Squares

The first training approach I focused on was *Ordinary Least Squares* (OLS). Since I am dealing with a regression task, where $y_t \in \mathbb{R}$ and the linear predictor is $h(x_t) = w^T x_t$, I evaluate the model using the *square loss*:

$$\ell(w) = \left(w^T x_t - y_t\right)^2 \tag{2}$$

The Empirical Risk Minimization (ERM) solution minimizes the residual sum of squares:

$$w_S = \text{argmin}_{w \in \mathbb{R}^d} \sum_{t=1}^{n} \left(w^T x_t - y_t\right)^2 = \text{argmin}_w \|Xw - y\|^2 \tag{3}$$

where X is the $n \times d$ design matrix (whose rows are x_t^T) and y is the target label vector.

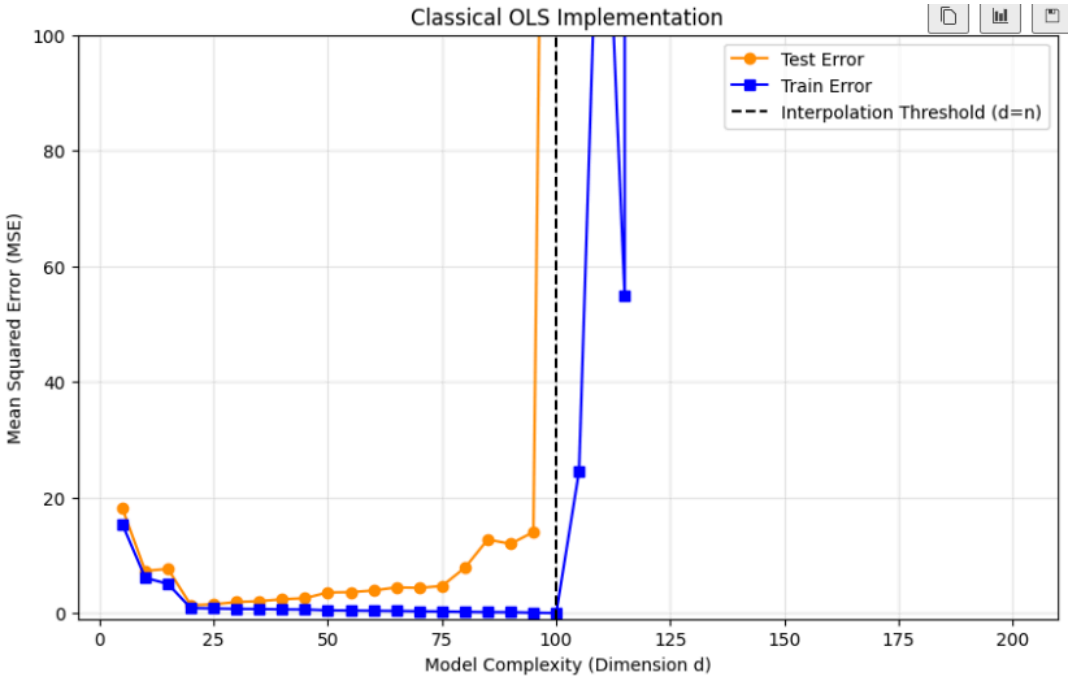
Since the objective function is convex, I can find the global minimum by setting its gradient with respect to w to zero:

$$\begin{aligned} \nabla_w \|Xw - y\|^2 = 2X^T(Xw - y) = 0 \\ X^T X w = X^T y \end{aligned} \tag{4}$$

If $X^T X$ is invertible (i.e., the features are linearly independent), we obtain the closed-form **Ordinary Least Squares (OLS)** solution:

$$w_S = \left(X^T X\right)^{-1} X^T y \tag{5}$$

As a first step, I implemented this mathematical formulation from scratch. By strictly applying the closed-form OLS solution, I obtained the following empirical results:



Analyzing the learning dynamics, the expected classical behavior is clearly visible across the different parameterization regimes. In the initial phase ($d < 20$), the model suffers from high bias. Because the true labels are generated using exactly 20 features, restricting the model to fewer dimensions deprives it of critical information needed to capture the underlying phenomenon, which naturally results in a high test error. However, at exactly $d = 20$, the model's capacity perfectly matches the true complexity of the data. Here, it gains access to all the necessary predictive information without any distracting noise, successfully reaching a "sweet spot" that minimizes the test error.

As the dimensionality d increases beyond 20, the model enters the high-variance regime. It starts leveraging the irrelevant features to blindly memorize the stochastic noise present in the training set. Consequently, the test error rapidly increases, while the training error decreases. This overfitting phase culminates at the interpolation threshold ($d = n = 100$), where the model perfectly memorizes the training data, driving the training error to absolute zero.

Finally, attempting to increase the features beyond this interpolation point ($d > n$) leads to a breakdown.

3.2. The Singularity Problem

In the previous phase of the work, I used the closed-form Ordinary Least Squares solution, which requires calculating the inverse of the covariance matrix:

$$(X^T X)^{-1} \tag{6}$$

From linear algebra, it is known that for a matrix to be invertible, it must be strictly full-rank (i.e., its rank must equal its dimension). The matrix $X^T X$ has dimensions $d \times d$, and its rank is bounded by the rank of the original $n \times d$ matrix X, which is at most $\min(n, d)$ (since a matrix's rank cannot exceed the number of rows or columns it has).

When the number of features d is less than or equal to n, the rank is d. The matrix is full-rank and therefore invertible. However, when the number of features strictly exceeds the number of samples ($d > 100$), the maximum possible rank is bounded by n. Since $n < d$, the matrix $X^T X$ becomes rank-deficient. Consequently, its determinant becomes exactly zero, making it a singular matrix.

Since the determinant is mathematically equivalent to the product of all eigenvalues, the presence of zero eigenvalues when $d > n$ is what drives the determinant to zero.

Mathematically, calculating the inverse involves dividing by the determinant:

$$(X^T X)^{-1} = \frac{1}{\det(X^T X)} \cdot \text{adj}(X^T X) \tag{7}$$

Since dividing by zero is undefined, the program should simply throw an exception for $d > 100$. However, the graph displays strange, erratic points beyond the threshold instead of an immediate crash. This happens due to floating-point precision issues: the computer approximates the theoretically zero-valued determinant (and the zero eigenvalues) as infinitesimally small numbers (e.g., 10^{-16}). As matrix inversion involves dividing by these near-zero eigenvalues, the computation avoids a crush but produces higly unstable and increasingly large weights.

To empirically verify that these anomalous points are just numerical artifacts and why the code does not immediately crash, I logged the minimum eigenvalue of the covariance matrix $X^T X$ and the maximum weight magnitude during the training loop:

```
...
d =  95 | Min Eigenvalue: 2.61e-01 | Max Weight: 2.12e+00
d = 100 | Min Eigenvalue: 1.03e-03 | Max Weight: 5.13e+00
d = 105 | Min Eigenvalue: 2.11e-15 | Max Weight: 5.42e+00
...
d = 120 | Min Eigenvalue: 8.93e-16 | Max Weight: 9.02e+02
...
d = 190 | Min Eigenvalue: 6.37e-16 | Max Weight: 4.25e+03
```

This output perfectly explains the program's behavior. Before the threshold ($d < 100$), the minimum eigenvalue is significantly greater than zero, allowing for a stable matrix inversion with bounded weights (around 2.4). However, as soon as $d > 100$, the minimum eigenvalue drops to floating-point zero (10^{-15}). Dividing by this microscopic approximation allows the code to keep executing without triggering a ZeroDivisionError, but it causes the weights to become progressively unstable as d increases (e.g., reaching 4.25×10^3 at $d = 190$).

Therefore, the erratic points plotted after $d = 100$ are not the result of standard overfitting, but merely the visual representation of this computational failure.

3.3. Ridge Regression

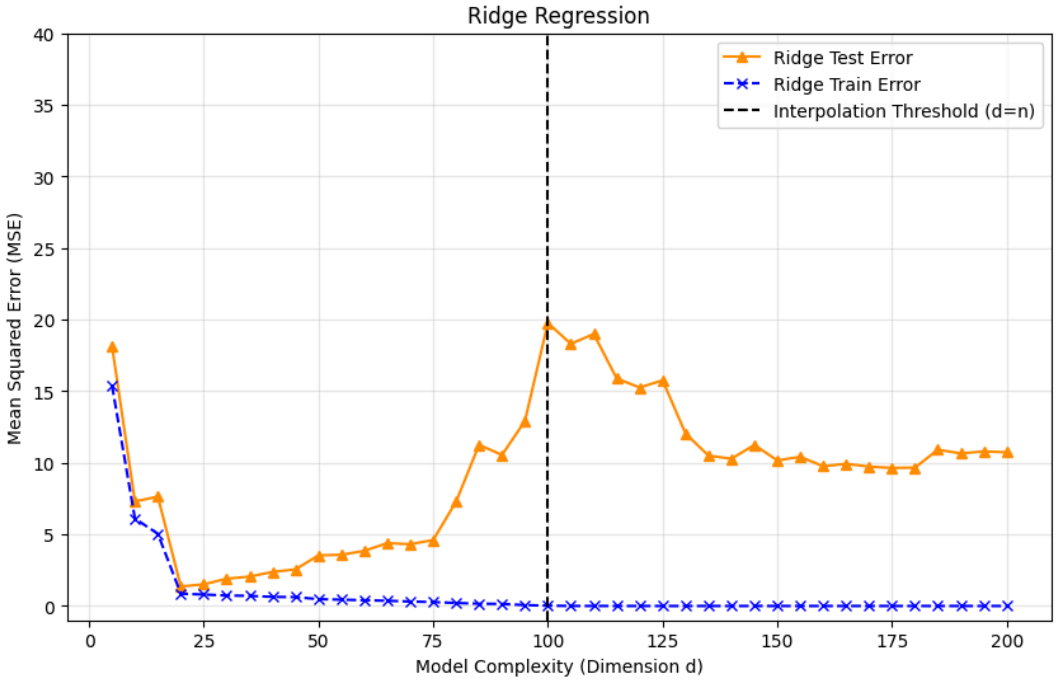
In real-world scenarios, it is common to encounter highly complex models with a massive number of features that must be trained on a much smaller number of examples ($d > n$). In these high-dimensional settings, classical unregularized ML fails, as we have seen above, because the matrix $X^T X$ becomes singular and not invertible.

To overcome this problem, a standard approach is to apply *Ridge Regression* (L_2 regularization). Ridge regression basically adds a small positive constant, α, to the main diagonal of the matrix:

$$w_{S,\alpha} = (X^T X + \alpha I)^{-1} X^T y \tag{8}$$

This mathematical addition ensures that all the eigenvalues of the matrix become strictly greater than zero. By doing so, it makes the matrix strictly positive-definite and always perfectly invertible, preventing the model from crashing.

To empirically verify this theoretical behavior, I ran the same experiment using Ridge Regression with a small penalty parameter ($\alpha = 0.1$).



As expected, the regularization term completely resolves the singularity breakdown. Looking at the graph, the massive error spike at the interpolation threshold ($d = n = 100$) has completely disappeared. Because the matrix inversion is now mathematically stable, the model no longer calculates artificially inflated weights. The test error remains safely bounded even when the number of features strictly exceeds the number of examples ($d > n$).

While Ridge Regression successfully prevents the model from crashing, it acts as a mathematical constraint. By penalizing the weights, it prevents the model from perfectly fitting the training data, forcing the model to accept some training error (bias) to prevent catastrophic overfitting (variance) [5].

However, this fails to explain the reality of modern machine learning. Today's highly complex models, such as large neural networks, operate heavily in the over-parameterized regime ($d \gg n$) often without relying on heavy explicit regularization like Ridge [6]. They are trained until they perfectly interpolate the training data (reaching exactly zero training error), yet they still generalize well to unseen data. Classical theory dictates that such models should strictly fail.

To explain this, we must analyze the Double Descent phenomenon.

4. Double Descent

To correctly recreate the double descent curve, I based my approach on the framework proposed by Belkin et al. [1], specifically focusing on their implementation for linear predictors.

Citing the paper:

> The minimum norm interpolating classifier… can be obtained directly by explicit norm minimization (solving an explicit system of linear equations), through SGD or by averaging trajectories.
>
> — [1]

Among these options, I chose to implement the *explicit norm minimization* from scratch.

In the over-parameterized regime ($d > n$), the model has more features than training examples. This creates an under-determined mathematical system: there are infinitely many different weight vectors w that can perfectly fit the training data (achieving $Xw = y$).

In accordance with the inductive bias described by Belkin et al., to decide which of these infinite solutions to use, I applied a mathematical version of Occam's razor: I selected the "simplest" model by finding the weights that are closest to zero. Mathematically, this means I want to satisfy the interpolation condition while strictly minimizing the L_2 norm of the weights:

$$\min \|w\|_2^2 \tag{9}$$

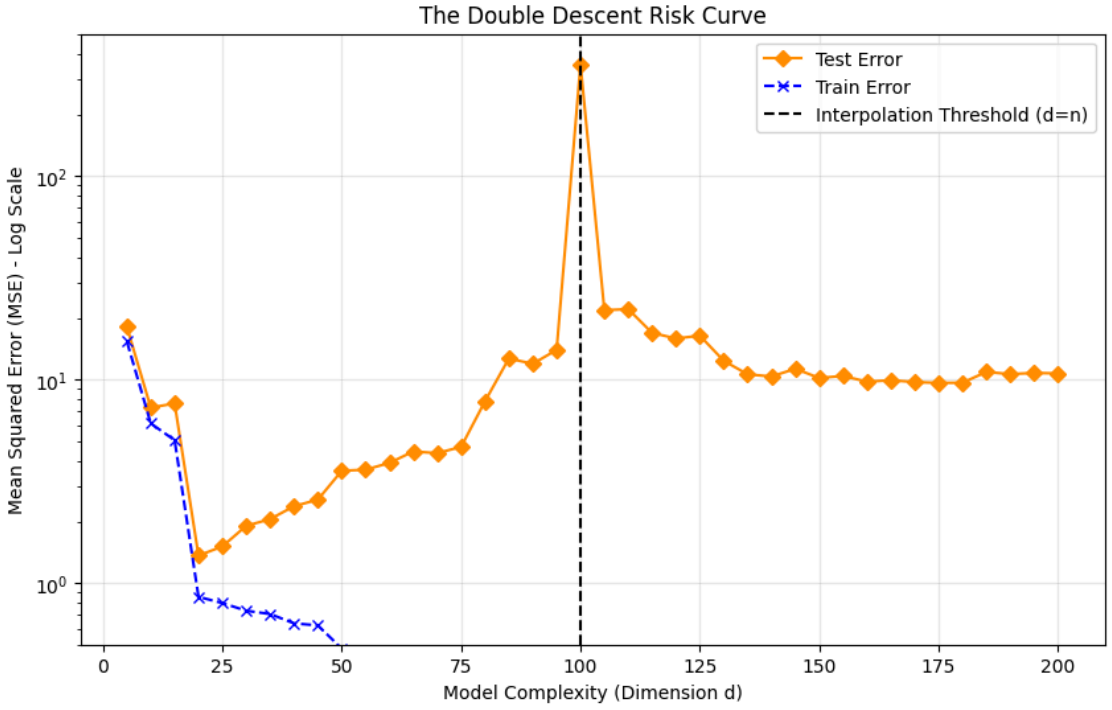
The norm is squared purely for mathematical and computational convenience. Finding the minimum of the squared Euclidean length yields the exact same optimal weights, but completely bypasses the complexity of differentiating a square root.

To implement this in practice, I researched the standard algebraic solution for under-determined systems [7].

Unlike the classical OLS formula, which tries to invert a $d \times d$ matrix (which becomes singular and non-invertible when $d > n$), the minimum norm solution flips the perspective. It solves the constrained problem: find w such that $Xw = y$ while minimizing $\|w\|_2^2$. This yields an $n \times n$ matrix (XX^T) instead. Since in this regime we have many features but very few examples, this smaller matrix stays "healthy" (full rank) and is easy to invert. By switching from a d-dimensional problem to an n-dimensional one, the math stays stable and provides the following closed-form solution:

$$w = X^T(XX^T)^{-1}y \tag{10}$$

Using this final formula, I trained the model once more, combining the classical OLS for the under-parameterized regime ($d \leq n$) with this explicit norm minimization for the over-parameterized regime ($d > n$). The resulting learning dynamics are shown below:



In this final graph, we can see exactly what happens after the interpolation threshold ($d = n$): the test error starts dropping again, creating the *second descent* in the over-parameterized regime ($d > n$). This confirms Belkin's theory and explains why modern models, like deep Neural Networks,

don't just fail when they have way more parameters than data points ($d \gg n$); instead, they actually generalize better.

The logic behind this drop is tied to the inductive bias of the algorithm. Since the model has so many extra degrees of freedom, it can find a solution that touches every training point but does so with smaller weights. This makes the final function less complex, leading it to ignore random noise and giving us the drop in test error we see.

To be sure that this "second descent" was actually driven by the model becoming simpler, I tracked the L2 norm of the weights throughout the experiment. Just as Belkin predicted, as the parameter space grows, the algorithm naturally picks progressively "smoother" solutions, which stabilizes the model's predictions on new data.

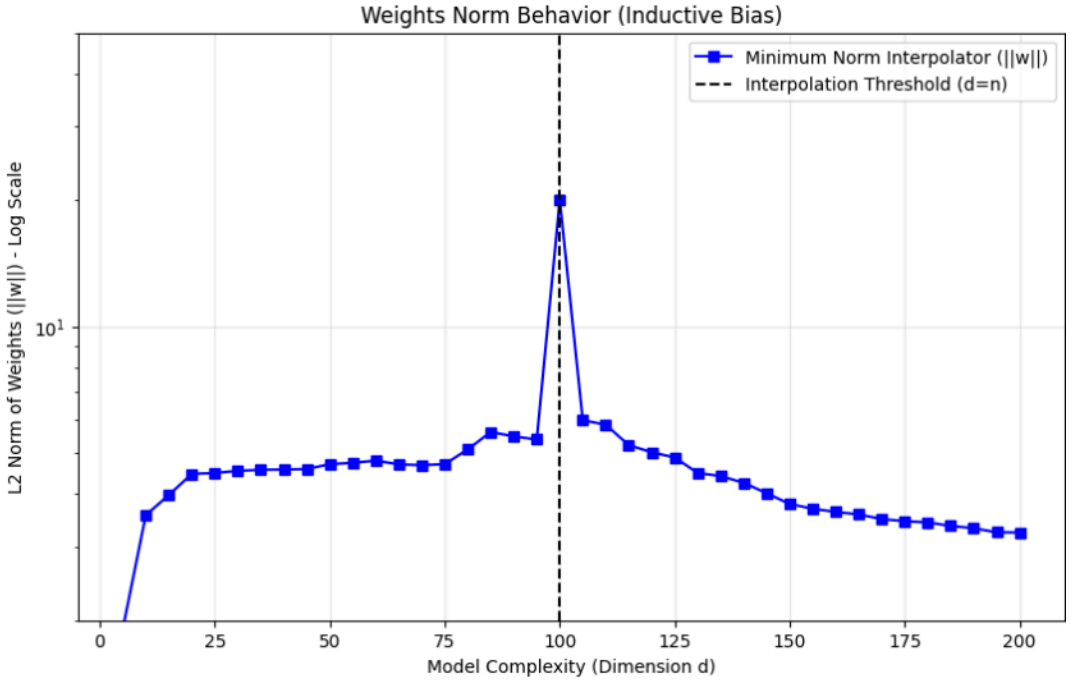


Figure 1: Empirical result: L_2 norm of weights

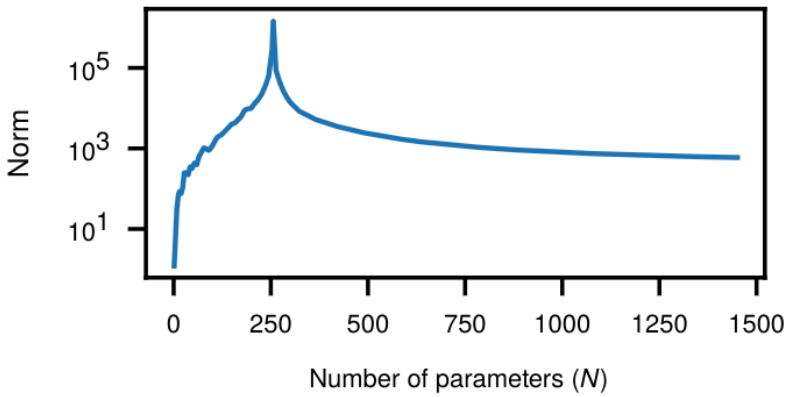


Figure 2: Belkin et al. [1], Fig. 10 (Norm)

Looking at the plot, (note the logarithmic scale on the Y-axis), the norm of the weights spikes exactly at the interpolation threshold ($d = n$). But once we move past that point, the norm steadily decreases as the number of features d grows. This curve matches the drop in test error almost perfectly. It's a great practical proof of the theory: having way more parameters actually helps the model generalize better, because it has enough "room" to find a simpler, smoother way to fit the data.

5. Extensions

5.1. Gradient Descent vs Closed-Form Solution

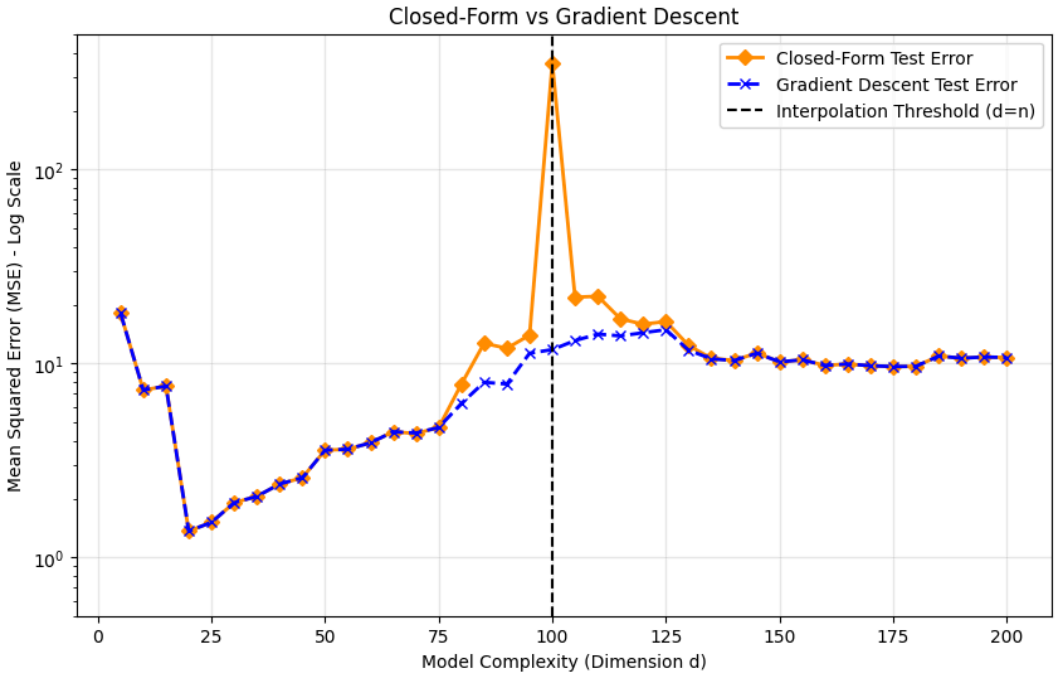
In the previous phase, I used a closed formula to find the minimum-norm solution for the $d > n$ regime. While this works in theory, inverting massive matrices in real-world scenarios is computationally expensive. A standard alternative is *Gradient Descent*.

Gradient descent updates the weights step by step to minimize the error. But since there are infinitely many perfect solutions in this over-parameterized regime, which one will it naturally find?

To test this, I implemented a simple Gradient Descent from scratch. Following the literature, I initialized all weights to zero ($w = 0$).

This starting point triggers a phenomenon called *implicit regularization* [8]. Without needing any complex matrix inversion, the Gradient Descent algorithm inherently acts as a regularizer. Specifically, Zhang et al. [6] demonstrated that for over-parameterized linear models, starting Gradient Descent at zero guarantees convergence to the exact minimum L_2-norm solution.

To verify this, I plotted the test error of my iterative Gradient Descent against my previous closed-form formula:



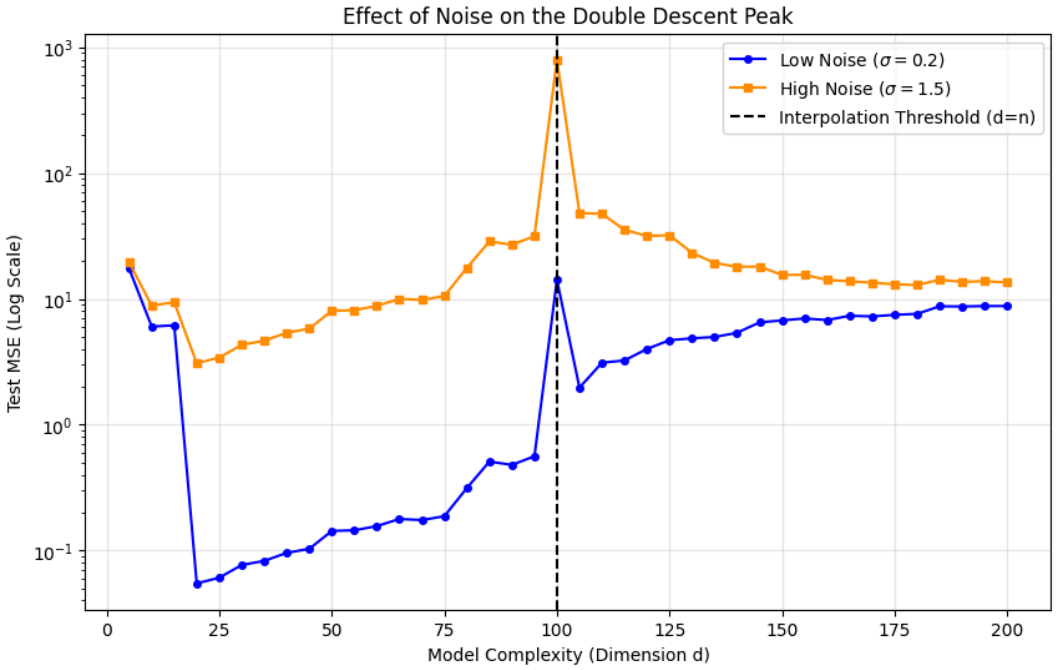
Looking at the graph, the two curves perfectly overlap in both the under-parameterized and highly over-parameterized regimes, confirming the implicit bias theory.

A divergence occurs exactly at the interpolation threshold ($d = n$). While the closed-form solution suffers from a massive error spike due to the inversion of a nearly singular matrix, the Gradient Descent curve remains significantly lower and more stable. This happens because Gradient Descent does not divide by zero; instead, it takes iterative steps. Running the algorithm for 5000 epochs with a learning rate of 0.01 prevents the weights from reaching the astronomically large values caused by the singularity.

5.2. Effect of Noise

To conclude the project, I explored how the intensity of noise in the training data affects the Double Descent curve.

By comparing a "clean" dataset ($\sigma = 0.2$) with a "noisy" one ($\sigma = 1.5$), I observed that the error spike at the interpolation threshold ($d = n$) becomes significantly more extreme as noise increases.



As shown in the plot, the high-noise curve (orange) remains consistently above the low-noise one (blue). As discussed in recent literature [9], this occurs because at the threshold $d = n$, the model is forced to perfectly fit every data point. At this critical point, there is precisely one solution to the system $Xw = y$, so the model must memorize all the noise in the training set. If these points contain high levels of noise, the weights must fluctuate wildly to interpolate them, leading to bad generalization.

I maintained the *logarithmic scale* for this visualization because the disparity between the two peaks is so large (nearly two orders of magnitude) that a linear scale would have made the low-noise curve appear flat, hiding its double descent.

6. Conclusions

At the end of their paper, Belkin et al. point out that while the classic U-shaped bias-variance curve has taught us a lot about machine learning, it has clear limits.

The results from my project confirm exactly that. The classical theory works perfectly fine when we have fewer features than examples (the under-parameterized regime). However, it completely misses the mark when it comes to explaining modern machine learning. By pushing past the interpolation threshold ($d > n$) and using the right inductive bias (like keeping the norm small), models don't just crash. Instead, they push through the singularity problem, experience a "second descent" in test error, and actually achieve great generalization.

Through the Gradient Descent analysis, I saw that we don't necessarily need a complex closed formula to get the best results. If the algorithm starts from zero, it naturally favors the simplest solution with the smallest weights. Furthermore, the experiment on the Effect of Noise showed that while dirty data makes the model struggle a lot at the critical threshold, having more parameters actually helps to stabilize the predictions and manage the noise better.

Bibliography

[1] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling modern machine-learning practice and the classical bias–variance trade-off," *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, pp. 15849–15854, 2019.

[2] scikit-learn developers, "Common pitfalls and recommended practices." Accessed: Apr. 22, 2026. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html°

[3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, Dec. 2012, doi: 10.1145/2382577.2382579°.

[4] J. Pineau *et al.*, "Improving Reproducibility in Machine Learning Research(A Report from the NeurIPS 2019 Reproducibility Program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021, [Online]. Available: http://jmlr.org/papers/v22/20-303.html°

[5] T. Hastie, "Ridge Regularization: An Essential Concept in Data Science," *Technometrics*, vol. 62, no. 4, pp. 426–433, 2020, doi: 10.1080/00401706.2020.1791959°.

[6] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," *arXiv preprint arXiv:1611.03530*, 2016.

[7] A. Rangamani, L. Rosasco, and T. Poggio, "For interpolating kernel machines, minimizing the norm of the erm solution minimizes stability," *arXiv preprint arXiv:2006.15522*, 2020.

[8] B. Neyshabur, R. Tomioka, and N. Srebro, "In search of the real inductive bias: On the role of implicit regularization in deep learning," *arXiv preprint arXiv:1412.6614*, 2014.

[9] I. Ullah and A. Welsh, "On the effect of noise on fitting linear regression models," 2024.